

1.1) The Language $L_1 = \{ \langle M \rangle \mid L(M) \subseteq \{a, aa\} \}$ is not in SD. A Turing machine cannot be built to show that only the values accepted by $L(M)$ are accepted while everything else is rejected therefore, L_1 is not decidable and therefore not in D. A reduction can be used to see if L_1 is in or not in SD.

$\neg H = \{ \langle M \rangle : M \text{ does not halt} \} \rightarrow L_1 = \{ \langle M \rangle \mid L(M) \subseteq \{a, aa\} \}$

Where $\neg H \leq L_1$

Let R be a mapping reduction from $\neg H$ to L_1 :

$R(\langle M, w \rangle) =$

- Construct $\langle M\# \rangle$ where $M\#(x)$:
 - if $x=a$ or $x=aa$ or both occur, accept
 - erase tape
 - write w on tape
 - run M on w
 - accept
- Return $\langle M\# \rangle$

If Oracle exists, $C = \text{Oracle}(R(\langle M, w \rangle))$ semi-decides L_1 :

C is correct:

- If $\langle M, w \rangle \in \neg H$: M doesn't halt and therefore $M\#$ halts on nothing. Oracle accepts.
- If $\langle M, w \rangle \notin \neg H$: M halts on w and therefore $M\#$ halts on everything. Oracle does not accept.

But, no machine can semi-decide $\neg H$ so Oracle cannot exist and therefore L_1 is not in SD.

1.2) The Language $L_2 = \{ \langle M \rangle \mid L(M) \supseteq \{a, aa\} \}$ is in SD. A reduction can be used to see if L_2 is in or not in D.

$H = \{ \langle M \rangle : M \text{ halts} \} \rightarrow L_2 = \{ \langle M \rangle \mid L(M) \supseteq \{a, aa\} \}$

Where $H \leq L_2$

Let R be a mapping reduction from H to L_2 :

$R(\langle M, w \rangle) =$

- Construct $\langle M\# \rangle$ where $M\#(x)$:
 - erase tape
 - write w on tape
 - run M on w
 - accept
- Return $\langle M\# \rangle$

If Oracle exists, $C = \text{Oracle}(R(\langle M, w \rangle))$ decides L_2 :

- R can be implemented as a Turing machine
- C is correct: $M\#$ accepts everything or nothing, depending on M halting.
- If $\langle M, w \rangle \in H$: M halts on w , therefore $M\#$ accepts all inputs, including a , aa , or both. Oracle accepts.
- If $\langle M, w \rangle \notin H$: M does not halt on w , so $M\#$ accepts nothing, in particular, it does not accept a , aa , or both. Oracle rejects.

But, no machine can decide H so Oracle cannot exist and therefore L_2 is not in D and is in SD.

1.3) The Language $L_3 = \{ \langle M \rangle \mid L(M) = \{a, aa\} \}$ is not in SD. A Turing machine cannot be built to show that only the values accepted by $L(M)$ are accepted while everything else is rejected therefore, L_3 is not decidable and therefore not in D. A reduction can be used to see if L_3 is in or not in SD.

$\neg H = \{ \langle M \rangle : M \text{ does not halt} \} \rightarrow L_3 = \{ \langle M \rangle \mid L(M) = \{a, aa\} \}$

Where $\neg H \leq L_3$

Let R be a mapping reduction from $\neg H$ to L_3 :

$R(\langle M, w \rangle) =$

- Construct $\langle M\# \rangle$ where $M\#(x)$:
 - if $x=a$ and $x=aa$ occur, accept
 - erase tape
 - write w on tape
 - run M on w
 - accept
- Return $\langle M\# \rangle$

If Oracle exists, $C = \text{Oracle}(R(\langle M, w \rangle))$ semi-decides L_3 :

C is correct:

- If $\langle M, w \rangle \in \neg H$: M doesn't halt and therefore $M\#$ halts on nothing. Oracle accepts.
- If $\langle M, w \rangle \notin \neg H$: M halts on w and therefore $M\#$ halts on everything. Oracle does not accept.

But, no machine can semi-decide $\neg H$ so Oracle cannot exist and therefore L_3 is not in SD.

1.4) The Language $L_4 = \{ \langle M \rangle \mid L(M) \in D \}$ is not in SD. A Turing machine cannot be built to show that only the values not in $L(M)$ are accepted while everything else is not accepted therefore, L_4 is not decidable and therefore not in D. A reduction can be used to see if L_4 is in or not in SD.

$H = \{ \langle M \rangle : M \text{ halts} \} \rightarrow L_4 = \{ \langle M \rangle \mid L(M) \in D \}$

Where $H \leq L_4$

Let R be a mapping reduction from H to L_4 :

$R(\langle M, w \rangle) =$

- Construct $\langle M\# \rangle$ where $M\#(x)$:
 - save x
 - erase tape write w on tape
 - run M on w
 - put x back on the tape
 - if $x \in SD$, accept, else reject
- Return $\langle M\# \rangle$

If Oracle exists, and decides L_4 then $C = \neg \text{Oracle}(R(\langle M, w \rangle))$ decides H :

C is correct:

- If $\langle M, w \rangle \in H$: $M\#$ checks x and accepts x if $x \in SD$. So $M\#$ accepts x , which is not Decidable. Oracle rejects. C accepts
- If $\langle M, w \rangle \notin H$: M does not halt on w . $M\#$ cannot accept anything. $L(M\#)$ is an empty set which is an element of D . Oracle accepts. C rejects.

But, no machine can decide H so Oracle cannot exist and therefore L_4 is not in SD.

1.5) The Language $L_5 = \{ \langle M \rangle \mid \neg L(M) \in D \}$ is not in SD. A Turing machine cannot be built to show that only the values not in $L(M)$ are accepted while everything else is not accepted therefore, L_5 is not decidable and therefore not in D. A reduction can be used to see if L_5 is in or not in SD. Keeping in mind the fact that the complement of a language that is an element of D is also still in D and vice versa, the same logic can be used as was used in 1.4 (above)

$H = \{ \langle M \rangle : M \text{ halts} \} \rightarrow L_5 = \{ \langle M \rangle \mid \neg L(M) \in D \}$

Where $H \leq L_5$

Let R be a mapping reduction from H to L_5 :

$R(\langle M, w \rangle) =$

- Construct $\langle M\# \rangle$ where $M\#(x)$:
 - save x
 - erase tape
 - write w on tape
 - run M on w
 - put x back on the tape
 - if $x \in SD$, accept, else reject
- Return $\langle M\# \rangle$

If Oracle exists, and decides L_5 then $C = \neg \text{Oracle}(R(\langle M, w \rangle))$ decides H :

C is correct:

- If $\langle M, w \rangle \in H$: $M\#$ checks x and accepts x if $x \in SD$. So $M\#$ accepts x , which is not Decidable. Oracle rejects. C accepts
- If $\langle M, w \rangle \notin H$: M does not halt on w . $M\#$ cannot accept anything. $L(M\#)$ is an empty set which is an element of D . Oracle accepts. C rejects.

But, no machine can decide H so Oracle cannot exist and therefore L_5 is not in SD.

1.6) The Language $L_6 = \{ \langle M \rangle \mid L(M) \in SD \}$ is in D Using the theorem stating that a language is in SD if and only if it is Turing-enumerable and vice versa. A Turing machine M can be constructed If L_6 as a subset of Σ^* is in SD, then there is a Turing machine that semi-decides L_6 . This means that the language of the TM that has it's acceptable language in SD would always have input that is either accepted or rejected (part of the semi-decidable Turing machine or not). Therefore a Turing machine M_b can be constructed for L_6 , making L_6 decidable. The Turing machine would entail the rules:

- If input rejects, halt and reject
- Else, halt and accept

and therefore L_6 is decidable, in D.

1.7) The Language $L_7 = \{ \langle M \rangle \mid \neg L(M) \in SD \}$ is in D Using the theorem stating that a language is in SD if is not Turing-enumerable. A Turing machine M can be constructed If $\neg L$ as a subset of Σ^* is in SD, then there is a Turing machine that semi-decides $\neg L$. This means that the language of the TM that has it's acceptable language in SD would always have input that is either accepted or rejected (part of the semi-decidable Turing machine or not). Therefore a Turing machine M_b can be constructed for L_7 , making L_7 decidable. The Turing machine would entail the rules:

- If input rejects, halt and reject
- Else, halt and accept

and therefore L_7 is decidable, in D.

1.8) The Language $L_8 = \{ \langle M \rangle \mid \text{there exists some input } w \text{ on which } M \text{ performs at least one right move} \}$ is in D. If the Turing machine reads any input then it would have to move right to read the next input value in the tape. However, the tape can also move left indefinitely. So a Turing machine can be constructed such that:

- if M moves rights, halt and accept
- if M halts without doing so, halt and reject

Making L_8 decidable and therefore in D.

1.9) The Language $L_9 = \{ \langle M \rangle \mid \text{there exists some input } w \text{ which } M \text{ accepts in } |w| \text{ steps or less} \}$ is in D. In $|w|$ steps, M cannot examine any more squares from its tape. So, running M on all strings in Σ^* , trying each for $|w|$ steps or until it halts such that:

- if at least one computation halts, accept
- else reject

Making L_9 decidable and therefore in D.

1.10) The Language $L_{10} = \{ \langle M_1, M_2 \rangle \mid \epsilon \in L(M_1) \cap L(M_2) \}$ is not in SD. No Turing machine can be created to accept if epsilon is apart of the intersection of two accepted languages and rejected otherwise, so L_{10} is not decidable and therefore not in D. A reduction can be used to see if L_{10} is in or not in SD.

$\neg H = \{ \langle M, w \rangle : M \text{ does not halt on } w \} \rightarrow L_{10} = \{ \langle M_1, M_2 \rangle \mid \epsilon \in L(M_1) \cap L(M_2) \}$

Where $\neg H \leq L_{10}$

Let R be a mapping reduction from $\neg H$ to L_{10} :

$R(\langle M, w \rangle) =$

- Construct $\langle M\# \rangle$ where $M\#(x)$:
 - erase tape
 - write w on tape
 - run M on w
 - accept
- construct $M?(X)$ that, on input x:
 - accept
- return $\langle M?, M\# \rangle$

If Oracle exists and semi-decides L_{10} , $C = \text{Oracle}(R(\langle M, w \rangle))$ semi-decides $\neg H$:

- R can be implemented as a Turing machine
- C is correct: $M?$ accepts everything including epsilon. $M\#$ accepts everything or nothing, depending on M halting on w.
- If $\langle M, w \rangle \in \neg H$: M doesn't halt on w, $M\#$ cannot get past $L(M\#)$ is an empty set. $L(M?) \cap L(M\#) = L(M?)$, which has epsilon. Oracle accepts.
- If $\langle M, w \rangle \notin \neg H$: M halts on w, so $L(M\#) = \Sigma$. The intersection of the languages does not contain epsilon. Oracle does not accept.

But, no machine can decide $\neg H$ so Oracle cannot exist and therefore L_{10} is not in SD.

2.1) Using Rice's Theorem proves L_1 is not in D, P is a nontrivial property of the SD languages such that the property $L(M) \subseteq \{a, aa\}$ is true if the language consists of a, aa, or both and is false for any other input. Then use the domain of P to show that the domain of P is the semi-decidable (in SD) languages, which applies in this case as the domain of P is the subset of languages accepted by the Turing machine.

2.2) Using Rice's Theorem proves L_2 is not in D, P is a nontrivial property of the SD languages such that the property $L(M) \supseteq \{a, aa\}$ is true if the language consists of a, aa, or both. Then use the domain of P to show that the domain of P is the semi-decidable (in SD) languages, which applies in this case as the domain of P is the of languages accepted by the Turing machine, including inputs a and aa.

2.3) Using Rice's Theorem proves L_3 is not in D, P is a nontrivial property of the SD languages such that the property $L(M) = \{a, aa\}$ is true if the language consists of only a and aa and is false otherwise. Then use the domain of P to show that the domain of P is the semi-decidable (in SD) languages, which applies in this case as the domain of P is the set of languages accepted by the Turing machine.

2.4) Using Rice's Theorem proves L_4 is not in D, P is a nontrivial property of the SD languages such that the property $L(M) \in D$ is true if the language consists of only inputs that are part of the D set and is false otherwise. Then use the domain of P to show that the domain of P is the semi-decidable (in SD) languages, which applies in this case as the domain of P is the set of languages accepted by some Turing machine. and we can prove that P is nontrivial:

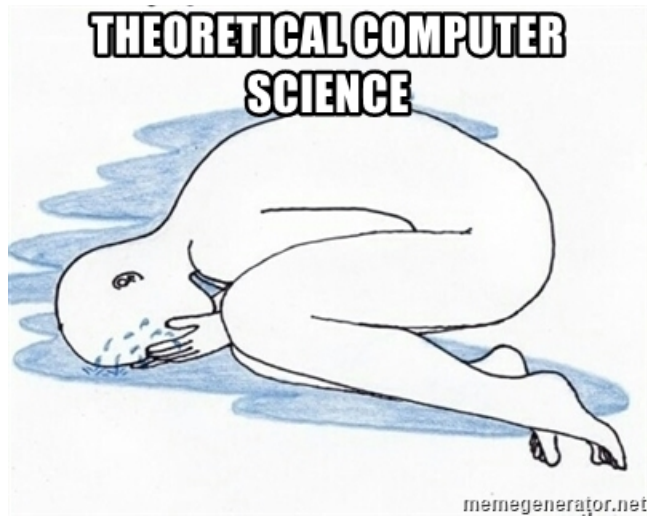
- $P(x \in D) = \text{true}$
- $P(x \notin D) = \text{False}$

2.5) Using Rice's Theorem proves L_5 is not in D, P is a nontrivial property of the SD languages such that the property $\neg L(M) \in D$ is true if everything outside of the language consists of only inputs that are part of the D set and is false otherwise. Then use the domain of P to show that the domain of P is the semi-decidable (in SD) languages, which applies in this case as the domain of P is the set of languages accepted by some Turing machine. and we can prove that P is nontrivial:

- $P(x \in D) = \text{true}$
- $P(x \notin D) = \text{False}$

2.10) Using Rice's Theorem proves L_{10} is not in D, P is a nontrivial property of the SD languages such that the property $\epsilon \in L(M_1) \cap L(M_2)$ is true if the intersection of the two languages has epsilon as an element and is false otherwise. Then use the domain of P to show that the domain of P is the semi-decidable (in SD) languages, which applies in this case as the domain of P is the set of languages accepted by this Turing machine. P can be proven as nontrivial as:

- $P(x = \epsilon) = \text{True}$
- $P(x \neq \epsilon) = \text{False}$



This course made me want to yeet myself into space. I need a nap. I am not editing this, so I am sorry for any inevitable spelling errors. Now I need to study this entire course so that I don't eat my exam paper in distress (ಠ_ಠ)